

软件工程基础第 1 章——软件工程概述

详细学习笔记

基于课堂课件整理

2026 年 5 月 28 日

目录

1 系统论与系统工程 (Systems Theory and Systems Engineering)	5
1.1 系统的概念 [p.4-5]	5
1.2 系统复杂性及其解决方案 [p.6]	5
1.3 系统工程的概念 [p.8-9]	5
1.4 系统工程典型实例：都江堰水利工程 [p.11-17]	6
1.4.1 采购阶段：需求分析 [p.12]	6
1.4.2 开发阶段：工程设计 [p.13]	6
1.4.3 运行阶段：维护管理 [p.14]	6
1.4.4 系统工程方法的体现 [p.15-17]	6
1.5 科学、技术与工程的关系 [p.18]	7
2 软件与软件工程 (Software and Software Engineering)	8
2.1 软件的定义与特点 [p.23]	8
2.2 软件的分类 [p.25-26]	8
2.2.1 按功能划分 [p.25]	8
2.2.2 按服务对象范围划分 [p.26]	8
2.3 软件危机 (Software Crisis) [p.28-29]	8
2.3.1 典型案例：IBM OS/360 [p.28]	8
2.3.2 软件危机的表现 [p.29]	9
2.4 软件工程的定义 [p.31-34]	9
2.4.1 各种定义 [p.32-34]	9
2.5 软件工程的发展阶段 [p.35-36]	9
2.5.1 三阶段划分 [p.35]	9
2.5.2 四阶段划分 [p.36]	10
2.6 软件产业的增长点 [p.37-39]	10
2.7 软件工程前沿研究方向 [p.40]	10

3 软件可信性 (Software Dependability)	11
3.1 可信性的概念 [p.45–47]	11
3.1.1 软件失效的典型案例	11
3.2 软件故障的类型 [p.48]	11
3.3 软件可信性的属性体系 [p.50]	11
3.4 可靠性 (Reliability) [p.51]	12
3.5 可用性 (Availability) [p.52]	12
3.6 防危性 (Safety) [p.53]	12
3.7 完整性 (Integrity) [p.54]	13
3.8 安全性 (Security) 与 CIA 三要素 [p.55]	13
3.9 可维护性 (Maintainability) [p.56]	13
3.10 软件可信性工程 [p.58–64]	13
3.10.1 故障处理的四种策略 [p.58]	14
3.10.2 故障避免 [p.59]	14
3.10.3 故障移除 [p.60]	14
3.10.4 软件容错 [p.61]	14
3.10.5 拜占庭容错 [p.62]	15
3.10.6 故障预测 [p.63]	15
3.10.7 可信性工程的主要任务 [p.64]	15
4 软件开发方法与技术 (Software Development Methods and Technologies)	16
4.1 软件工程的四个核心内容 [p.69–71]	16
4.2 软件工程的基本原理 [p.73]	16
4.3 软件开发方法概述 [p.78–79]	17
4.4 结构化方法 (Structured Method) [p.81–85]	17
4.4.1 核心思想 [p.81]	17
4.4.2 结构化方法的起源: Dijkstra 的贡献 [p.82]	18
4.4.3 结构化方法发展史 [p.83]	18
4.4.4 结构定理 (Böhm-Jacopini 理论) [p.84]	18
4.5 面向对象方法 (Object-Oriented Method) [p.86–90]	19
4.5.1 核心思想 [p.86]	19
4.5.2 面向对象方法的不足 [p.87]	19
4.5.3 面向对象方法的起源 [p.88]	19
4.5.4 问题空间与软件空间一致 [p.89]	20
4.6 面向方面方法 (Aspect-Oriented Method) [p.91–96]	20
4.6.1 横切关注点 (Crosscutting Concerns) [p.91–92]	20
4.6.2 面向方面程序设计的一般模型 [p.93]	20
4.6.3 关注点的五类划分 (Ian Sommerville) [p.94]	20
4.6.4 面向方面方法概述 [p.95]	21
4.7 形式化方法 (Formal Method) [p.97–100]	21

4.7.1	核心思想 [p.97]	21
4.7.2	发展历史 [p.98–99]	21
4.8	多方法集成: SOFL 方法 [p.101–103]	22
5	软件工具与集成开发环境 (Software Tools and Integrated Development Environments)	23
5.1	软件工具 [p.108–112]	23
5.1.1	软件工具的局限性 [p.112]	23
5.2	集成型软件开发环境 [p.114–116]	23
5.2.1	集成化项目支持环境 (IPSE) [p.115]	23
5.2.2	ECMA 软件开发环境参考模型 [p.116]	24
5.3	AI 辅助软件工具 [p.118–127]	24
5.3.1	大模型如何使能软件工程 [p.118]	24
5.3.2	生成式 AI 核心技术 [p.119]	24
5.3.3	AI 编程工具 [p.121]	24
5.3.4	百度千帆 API 调用示例 [p.123–125]	25
5.3.5	AI 辅助编码环境搭建 [p.126]	25
5.4	Flask Web 框架实战 [p.128–193]	25
5.4.1	Flask 框架特点 [p.128]	25
5.4.2	环境搭建 [p.130]	25
5.4.3	第一个 Flask 应用 [p.131]	26
5.4.4	项目目录结构 [p.133]	26
5.4.5	配置管理 [p.134]	27
5.4.6	依赖管理 [p.135]	27
5.4.7	路由机制 [p.138–139]	27
5.4.8	动态路由 [p.140]	27
5.4.9	HTTP 请求方法 [p.141]	27
5.4.10	文件上传 [p.142–143]	28
5.4.11	模板 (Templates) 与 Jinja2 [p.145–154]	28
5.4.12	Bootstrap 集成 [p.155–158]	28
5.4.13	Favicon 设置 [p.159]	29
5.4.14	Web 表单 (Flask-WTF) [p.161–167]	29
5.4.15	数据库操作: Flask-SQLAlchemy [p.168–192]	29
6	基于 Trae 的软件开发 (Trae-based Software Development)	32
6.1	Trae 工具简介 [p.194–199]	32
6.1.1	Trae 的安装与基本使用 [p.198–199]	32
6.2	Trae 智能体简介 [p.200–205]	32
6.3	Skill 创建与使用 [p.206–213]	33
6.3.1	Skill 简介 [p.207]	33

6.3.2	Skill 使用场景 [p.208]	33
6.3.3	Skill 创建示例 [p.209–210]	33
6.3.4	Skill 使用 [p.211–212]	34
6.3.5	Skill 文件结构 [p.213]	34
6.4	Rule 创建与使用 [p.214–224]	34
6.4.1	Rule 简介 [p.215]	34
6.4.2	个人 Rule [p.216–218]	35
6.4.3	项目 Rule [p.219–221]	35
6.4.4	Rule 最佳实践 [p.222–224]	35
7	附录：核心概念速查表	37

1 系统论与系统工程 (Systems Theory and Systems Engineering)

1.1 系统的概念 [p.4-5]

客观世界是一个统一的整体，具有系统性。中国古代哲学中的阴阳五行学说（金、水、木、火、土五种基本元素，以及阴阳两种相互作用的力量）体现了朴素的系统观 [p.4]。古希腊唯物主义哲学家德谟克利特（Democritus）提出“原子论”，认为原子组成万物，形成不同系统层次的世界 [p.4]。

系统 (System) 是由两个或以上有机联系、相互作用的要素 (Elements) 所组成，具有特定功能、结构和环境的整体 [p.5]。系统的三个核心构成要素为：

- **功能 (Function)**：系统存在的作用和价值，体现系统与外部的相互作用。
- **结构 (Structure)**：反映组成系统的诸要素之间相对稳定的相互联系方式（组成要素）。
- **环境 (Environment)**：系统边界之外，与系统产生相互作用和相互影响的部分（依存关系）。

1.2 系统复杂性及其解决方案 [p.6]

现实系统往往规模庞大、十分复杂。系统复杂性体现在多个维度 [p.6]：

- 要素多样性（要素多维）
- 动态变化性（动态变化）
- 功能多样性（功能多样）
- 人机交互性（人机交互）
- 开放适应性（开放适应）
- 学习重组能力（学习重组）

解决系统复杂性的根本方法是**系统工程 (Systems Engineering)** [p.6]。

1.3 系统的概念 [p.8-9]

系统工程是从整体出发，合理开发、管理、运行和改进一个大规模复杂系统所需的思想、理论、方法和技术的总称 [p.8]。它是组织管理系统的规划、研究、设计、制造、试验和使用的科学方法，是一种对所有系统都具有普遍意义的方法 [p.8]。

系统工程以大规模复杂系统为研究对象，在运筹学 (Operations Research)、系统理论 (Systems Theory)、管理科学 (Management Science) 等的理论指导下，按照一定目的进行系统的采购、开发与运行，通过定量和定性相结合，达到总体效果最优 [p.9]。

系统工程活动包括 [p.9]：

1. **采购 (Acquisition)**：决定采购范围、预算、进度及系统高层需求。

2. **开发 (Development)**: 通过需求分析、系统设计、子系统开发、系统集成、系统测试和系统部署等活动来得到最终系统。
3. **运行 (Operation)**: 用户按系统定义的目的来使用系统。

1.4 系统工程典型实例：都江堰水利工程 [p.11–17]

都江堰水利工程位于四川成都平原西部，距成都 56 公里。公元前 256 年，战国时期秦国蜀郡太守李冰率众修建 [p.11]。它是系统工程的古代典范，体现了系统工程方法的精髓。

1.4.1 采购阶段：需求分析 [p.12]

李冰到任后，带领儿子二郎跋山涉水，多次沿岷江岸进行实地考察 [p.12]。岷江发源于岷山南麓，全长 1279 千米，年均径流量 900 多亿立方米，是黄河的两倍多。问题背景为 [p.12]：

- 成都平原地形西北高东南低。
- 春夏山洪爆发时，江水奔腾而下进入成都平原，由于河道狭窄，常常引起洪灾。
- 岷江东岸的玉垒山阻碍了江水东流，造成东旱西涝。

高层需求：与自然和谐共存的基础上，满足防洪、灌溉、清淤三项需求 [p.12]。

1.4.2 开发阶段：工程设计 [p.13]

都江堰工程由三大主体工程组成 [p.13]：

- **鱼嘴（分水堤）**：将岷江分为内江和外江，实现分水功能。
- **飞沙堰（溢洪道）**：排沙功能，利用弯道环流原理自动分级排沙。
- **宝瓶口（引水口）**：引水功能，控制进入成都平原的流量。

1.4.3 运行阶段：维护管理 [p.14]

“岁修”对都江堰的环境保护和持续发展具有重要意义 [p.14]：

- “淘滩”：淘挖淤积于内江、外江进水口河床的沙砾卵石。
- “作堰”：修复包裹鱼嘴的金刚堤、飞沙堰以及后来修建的人字堤等堰堤。

1.4.4 系统工程方法的体现 [p.15–17]

都江堰体现了系统工程的核心原则 [p.15]：

- **总体目标最优化**：实现“分四六、平潦旱”（枯水期内江六成、外江四成，进行灌溉；丰水期内江四成、外江六成，避免洪灾） [p.16]。
- **选址最优**：利用地形。

- 就地取材。
- 自动分级排沙、自动调节水量。
- 造价低廉、维护方便。

技术创新方面 [p.17]：李冰采用“火烧水浇”的方式，利用热胀冷缩的原理使岩石迸裂疏松，历经八年，在玉垒山凿出了一个宽 20 米、高 40 米、长 80 米的山口，取名为“宝瓶口”。

1.5 科学、技术与工程的关系 [p.18]

陈清泉院士对科学、技术、工程三者的关系给出了精辟论述 [p.18]：

- **科学 (Science)**：以发现为核心，是对自然的本质及其运行规律的探索、发现和揭示，并归纳为真理。科学家的探索往往是出于好奇心，并没有明确的实用目的。
- **技术 (Technology)**：以发明为核心，是改善人类社会生活的手段，可以是方法、装置、工具、仪器仪表、过程，讲求的是技巧。
- **工程 (Engineering)**：集成科学和技术，来解决实际问题。工程是利用科学原理和技术在一定边界条件下进行集成优化和综合优化，有目的地完成设计、构建、运行等项目（涉及人、财、物）。

2 软件与软件工程 (Software and Software Engineering)

2.1 软件的定义与特点 [p.23]

软件 (Software) 是计算机系统中看不见、摸不着的逻辑部分, 以程序 (Program)、数据 (Data) 和文档 (Documentation) 的形式出现 [p.23]。软件具有以下独特特点:

- 软件不是“制造”产生的, 而是“研发”出来的: 这导致软件项目管理和软件产品保护困难。
- 软件不会被“用坏”: 但软件维护困难 (软件维护不能通过重复制造解决), 需要持续修改和升级。
- 软件大多是“定制”的: 导致软件开发的质量和效率受到影响。
- 软件成本难于估计: 导致软件项目计划失效。

2.2 软件分类 [p.25–26]

2.2.1 按功能划分 [p.25]

- **系统软件 (System Software):** 与计算机系统硬件紧密交互, 协调计算机系统各部分工作的软件。例如操作系统 (OS)、设备驱动程序等。
- **支撑软件 (Support Software):** 协助使用者开发软件的工具性软件。例如程序编译器、自动化测试软件等。
- **应用软件 (Application Software):** 为使用一个计算机系统以得到某种功能而专门开发的软件。例如多媒体播放软件、微波炉的按钮控制软件等。

2.2.2 按服务对象范围划分 [p.26]

- **项目软件 (Project Software):** 软件开发机构受特定用户委托开发而成的软件, 软件描述通常由委托用户给出。例如空中交通管制系统。为了争取开发合同, 必须重视质量管理。
- **产品软件 (Product Software):** 软件开发机构直接为市场开发的软件, 软件描述由开发机构给出。例如游戏软件。为了争取市场, 必须重视功能、性能、价格和售后服务。

2.3 软件危机 (Software Crisis) [p.28–29]

2.3.1 典型案例: IBM OS/360 [p.28]

1961 年底, IBM 开始实施“360 系统电子计算机计划”, 整个计划投资约需 50 亿美元 (作为对比, 曼哈顿工程为 20 亿)。360 操作系统的开发用了 5000 个人年 (person-years), 软件工程师超过 2000 人 (作为对比, Windows 2000 动用了 1700 名), 花费超过 5 亿美元, 最终也没能完全实现当初的设想 [p.28]。该项目负责人 Frederick Brooks 据此撰写了著名的《人月神话》(*The Mythical Man-Month*) [p.28]。

2.3.2 软件危机的表现 [p.29]

20 世纪 40 年代中期到 60 年代中期，软件开发具有明显的个体化特征。20 世纪 60 年代中期到 70 年代中期进入“软件作坊”阶段，软件危机开始显现 [p.29]：

- 软件质量差，可靠性难以保证（bug 问题）。
- 成本难以控制，很少有在预算内完成的。
- 开发进度难以把握，周期拖得很长。
- 可维护性较差，维护人员和费用不断增加。

2.4 软件工程的定义 [p.31–34]

在 1968 年 NATO（北大西洋公约组织）的计算机科学家在联邦德国召开国际会议，讨论软件危机问题，正式提出了“软件工程”（Software Engineering）这一术语 [p.31]。

2.4.1 各种定义 [p.32–34]

- **NATO 会议定义** [p.32]：软件工程是用来建立和使用合理的工程原则，以经济地获取可靠的、且在真实机器上可高效工作的软件。
- **计算机百科全书定义** [p.32]：应用计算机科学、数学及管理科学等原理，以工程化方法制作软件的工程。它借鉴传统工程的原则、方法，创建软件以达到提高质量、降低成本的目的。
- **综合理解** [p.33–34]：
 - **性质（能力）**：指导软件开发和维护的工程性学科。
 - **理论基础**：计算机科学（Computer Science）、管理科学（Management Science）和数学（Mathematics）等。
 - **手段**：采用工程化的概念、原理、技术和方法进行软件的开发和维护，把经过时间证明正确的管理措施和当前能够得到的最好的技术、方法相结合。
 - **目的**：以期用较少的代价获取高质量的软件。

2.5 软件工程专业的发展阶段 [p.35–36]

2.5.1 三阶段划分 [p.35]

1. **软硬一体化阶段（–1975 年）**：软硬件一起购买，软件作为硬件的附属。
2. **软件产品化、产业化阶段（1975 年–1995 年）**：软件单独购买，形成独立产业。
3. **软件网络化、服务化阶段（1995 年–现在）**：不买软件买服务，SaaS（Software as a Service）模式兴起。

2.5.2 四阶段划分 [p.36]

1. **程序设计阶段（20 世纪 50 至 60 年代）**：专家的个体手工劳动；程序的写法可以不受任何限制；软件受制于硬件的发展。
2. **程序系统阶段（20 世纪 60 至 70 年代）**：软件作坊；开始注重程序设计风格；软件供不应求——软件危机（内因：软件本身的特点；外因：软件开发和维护以及组织管理不当）。
3. **软件工程阶段（20 世纪 70 年代至今）**：工程化、多学科交叉。
4. **智能软件工程阶段（21 世纪 20 年代至今）**：自动化，AI 赋能软件工程。

2.6 软件产业的增长点 [p.37–39]

- **云计算（Cloud Computing）**：提供算力资源 [p.37]。
- **物联网（Internet of Things, IoT）**：产生大数据（Big Data）[p.38]。
- **人工智能（Artificial Intelligence, AI）**：模型 + 数据 + 算力，生成式 AI（Generative AI）成为新热点 [p.39]。

2.7 软件工程前沿研究方向 [p.40]

调研报告要求（2000-3000 字，三次迭代增量方式撰写）[p.40]：

- 群智化软件工程（Crowd-based Software Engineering）
- 开发运维一体化软件工程（DevOps Software Engineering）
- 云计算软件工程（Cloud Computing Software Engineering）
- 智能化软件工程（Intelligent Software Engineering）
- 大数据驱动软件工程（Big Data-driven Software Engineering）
- 区块链软件工程（Blockchain Software Engineering）

3 软件可信性 (Software Dependability)

3.1 可信性的概念 [p.45–47]

可信性 (Dependability) 反映了一个系统向其用户提供预期服务水平的能力 [p.45]。

3.1.1 软件失效的典型示例

1. **阿里亚娜 V 型火箭爆炸 (1996 年)** [p.45]: 1996 年 6 月 4 日, 阿里亚娜 V 型火箭升空约 30 秒后姿态控制失灵, 随后解体并引发了剧烈的爆炸, 造成 5 亿欧元损失。失败原因是火箭导航的软件系统发生失效——存放火箭水平速率的值出现溢出 (制导系统需要将测得的速度数据从 64 位浮点数格式转换为 16 位有符号整数格式)。
2. **爱国者导弹拦截失败 (1991 年)** [p.46]: 海湾战争期间, 美国爱国者导弹系统未能成功拦截伊拉克飞毛腿导弹。错误是由于系统内置时钟长时间运行后产生微小误差, 误差累计导致雷达系统无法准确预测飞毛腿导弹的弹道。根本原因是十进制的 $1/10$ 用二进制表示会产生微小精度误差。
3. **12306 订票系统崩溃** [p.47]: 故障根本原因在于系统架构规划以及客票发放机制存在缺陷, 无法支持大并发量的交易。

3.2 软件故障的类型 [p.48]

软件失效往往是由于软件系统错误状态引起。引起软件系统错误状态的原因称为**软件故障 (Software Fault)** [p.48]:

- **退化故障 (Degradation Fault) / 耗损故障**: 由于产品的规定性能随时间增加而逐渐衰减引起。软件故障不是退化故障。
- **设计故障 (Design Fault)**: 系统设计中存在的缺陷。设计故障在软件生命周期的任何时刻都会出现。
- **拜占庭故障 (Byzantine Fault)**: 一类行为完全无法预测的故障, 对系统不同部分影响不同。由于恶意攻击和软件错误会导致故障节点表现出拜占庭故障。

3.3 软件可信性的属性体系 [p.50]

软件可信性是一系列软件质量属性的集合 [p.50]:

- 可靠性 (Reliability)
- 可用性 (Availability)
- 防危性 (Safety)
- 完整性 (Integrity)
- 安全性 (Security)

- 可维护性 (Maintainability)
- 信任 (Trust)

3.4 可靠性 (Reliability) [p.51]

软件可靠性是对软件给定条件下在规定的时间内持续提供正确服务的一种概率度量 $R(t)$ [p.51]:

- 假定系统从第 0 时刻开始正确提供服务, $R(t)$ 表示在 $[0, t]$ 的时间段内在规定的运行环境中软件系统正确运行的概率。
- 给定条件: 包括软件运行的软硬件环境以及软件运行的输入空间及其概率分布。
- 规定的时间: 一般分为日历时间、执行时间或时钟时间三种。
- 失效的概率 $Q(t) = 1 - R(t)$ 。
- 关键特点: 软件缺陷导致的失效往往没有先兆, 一些微小的变化也可能引起失效。

3.5 可用性 (Availability) [p.52]

软件可用性 $A(t)$ 是指软件系统在 t 时刻在一定条件下能够正确运行的概率 [p.52]。可靠性要求系统提供连续的服务, 而可用性允许系统失效, 在经过维修后仍能正确运行即可。

关键指标:

- **MTTF (Mean Time To Failure, 平均失效等待时间):** 系统从恢复运行到下一次失效的平均时间。
- **MTTR (Mean Time To Repair, 平均失效修复时间):** 从失效到修复的平均时间。
- 稳态可用度:

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

- **最大修复时间 MaxTTR:** 以概率 q 不超过时间 T , 即 $P(\text{MaxTTR} \leq T) = q$ 。

3.6 防危性 (Safety) [p.53]

软件防危性定义为单位时间内软件失效损失的期望值小于规定的阈值 [p.53]。对于安全攸关的系统 (Safety-Critical Systems), 必须在系统开发早期阶段就将防危性考虑在内。

风险 (Risk) 计算公式:

$$\text{Risk} = \sum_i P_i(\text{失效}) \times L_i(\text{失效})$$

其中 $P_i(\text{失效})$ 表示单位时间第 i 种失效出现的概率, $L_i(\text{失效})$ 表示第 i 种失效出现带来的损失。

3.7 完整性 (Integrity) [p.54]

软件完整性是指对软件系统不存在未经授权的修改以及对软件系统行为不存在未经授权的操控 [p.54]:

- **静态完整性**: 软件本身对应的程序代码不被非授权篡改。
- **动态完整性**: 软件运行过程中不存在对软件行为的未经授权的操控。

完整性保护的三类技术:

1. **预防技术**: 用来防止软件受到攻击 (例如登录口令)。
2. **检测技术**: 在攻击无法避免的情况下检测攻击正在发生或已经发生, 并向用户报告攻击的性质和严重程度 (例如输入密码连续错误三次锁定)。
3. **恢复技术**: 通过容错或备份等处理技术来阻止攻击, 使系统保持正常运行。

3.8 安全性 (Security) 与 CIA 三要素 [p.55]

CIA 三要素: 机密性 (Confidentiality)、完整性 (Integrity)、可用性 (Availability) 三者统称为软件安全性三要素 [p.55]:

- **机密性 (Confidentiality)**: 又称隐私。信息只能在所授权的时间、所授权的地点暴露给所授权的个人、机构或计算机软件程序, 对于未被授权者, 这些信息是不可获得或不可理解的。
- **安全性 vs 防危性**: 安全性 (Security) —— 遵守交规, 事故发生前发生作用; 防危性 (Safety) —— 买保险, 事故发生后发生作用。

3.9 可维护性 (Maintainability) [p.56]

软件可维护性 $M(t)$ 是指软件失效后在一段给定的时间内可修复的概率 [p.56]:

$$M(t) = 1 - e^{-\mu t}$$

其中 μ 为系统的维修率。

影响软件可维护性的因素:

- **可理解性 (Understandability)**: 理解软件的结构、接口、功能和内部过程的难易程度。
- **易分析性 (Analyzability)**: 诊断软件故障, 并对需修改的部分进行定位的难易程度。
- **可修改性 (Modifiability)**: 软件便于修改的难易程度。
- **易测试性 (Testability)**: 为程序建立验证准则和性能评价的难易程度。

3.10 软件可信性工程 [p.58–64]

软件可信性工程是指在软件开发过程中利用有关方法和技术确保最终系统满足可信性需求 [p.58]。

3.10.1 故障处理的四种策略 [p.58]

1. **故障避免 (Fault Avoidance)**: 在软件开发和维护过程中避免引入故障。
2. **故障移除 (Fault Removal)**: 检测、识别和修复故障。
3. **容错 (Fault Tolerance)**: 出现故障时, 系统仍能继续提供可接受服务的能力。
4. **故障预测 (Fault Prediction)**: 预测将来可能出现的故障种类、数量和故障对于系统的影响。

3.10.2 故障避免 [p.59]

在软件开发和维护的各个阶段, 都可能引入故障 [p.59]:

- 严格按照工程的方法来开发和维护软件 (掌握工程方法、遵循工程方法)。
- 原型技术可以避免需求规格说明不完整。
- 形式化方法有助于软件正确性验证。
- 制定并遵循一致的编码标准和规范提高可维护性。
- 重视培训避免使用不当。

3.10.3 故障移除 [p.60]

故障移除是在故障引入之后进行的处理 [p.60], 包括:

- 人工审核技术 (Manual Review)
- 软件测试技术 (Software Testing)
- 模型检测技术 (Model Checking)
- 系统仿真技术 (System Simulation)

3.10.4 软件容错 [p.61]

容错就是要避免故障引起不可接受的失效 [p.61]。软件容错处理复杂的原因:

- 不知道故障的表现形式。
- 难以确定故障在软件中的位置。
- 不清楚故障的失效语义。
- 设计故障的影响很难估计。

冗余 (Redundancy) 是软件容错处理的常用方法:

- **时间冗余 (Time Redundancy)**: 重复计算或数据传输多次并对结果进行比较。
- **空间冗余 (Space Redundancy)**: 通过增加多余的部件或数据项和表决器, 降低故障引起的失效。

3.10.5 拜占庭容错 [p.62]

拜占庭容错 (Byzantine Fault Tolerance, BFT) 是系统在出现拜占庭故障的情况下仍能够保持正确处理的容错技术 [p.62]。

拜占庭将军问题 (Byzantine Generals Problem): 考虑 4 个将军的情况, 假设 4 个将军中最多只有 1 个背叛者。当 4 个将军 A、B、C、D 把敌人包围后, 必须协商一个统一的时间发起进攻。对于忠诚的将军来说, 他不知道谁是背叛者, 所以他不能完全相信接收到的命令, 必须对命令做出判断 [p.62]。

解决问题的核心思想: 对于每一个收到命令的将军, 都要去询问其他人, 他们收到的命令是什么 [p.62]。

3.10.6 故障预测 [p.63]

软件故障预测是通过评估系统的行为、结构等预测将来可能出现的故障种类、数量和故障对于系统的影响 [p.63]:

- **定性故障预测:** 对导致系统失效的模式或事件组合进行排序。
- **定量故障预测:** 采用概率来评估某些可信性属性满足的程度。
- **静态故障预测:** 基于故障相关的软件度量数据, 对故障的数量或者分布进行预测。
- **动态故障预测:** 基于故障或者失效产生的时间, 对故障随时间的分布进行预测。

3.10.7 可信性工程的主要任务 [p.64]

1. 针对系统的功能需求, 确定系统可能遭受的风险。
2. 针对每个风险, 确定导致该风险的故障。
3. 针对每个需要处理的故障确定可信性需求。
4. 采用四种故障处理技术来处理软件故障。
5. 对系统是否满足可信性需求进行评估。
6. 重复上述任务, 确保软件系统尽可能满足系统的可信性需求。

4 软件开发方法与技术 (Software Development Methods and Technologies)

4.1 软件工程的四个核心内容 [p.69–71]

软件工程主要有四个方面的内容，软件工程管理则贯穿于这四个方面 [p.69]:

1. **标准和规范 (Standards and Specifications):** 使各种工作有章可循，进而提高生产效率和产品质量。
2. **过程与模型 (Processes and Models):** 过程是活动和任务，模型将过程有机结合。
3. **方法和技术 (Methods and Technologies):** 方法是看待软件系统的立场和观点，技术则是方法的具体实现。
4. **工具和环境 (Tools and Environments):** 工具为软件开发和维护提供自动的或半自动的支持，环境是工具的有机集成。

这四个研究内容构成了以软件质量 (Software Quality) 为核心的层次结构 [p.71]:

工具和环境 (Tools & Environments)
方法和技术 (Methods & Technologies)
过程与模型 (Processes & Models)
标准和规范 (Standards & Specifications)
软件质量 (Software Quality)

4.2 软件工程的基本原理 [p.73]

确保软件产品质量和开发效率的原理的最小集合 [p.73]:

1. 用分阶段的生存周期计划严格管理。
2. 坚持进行阶段评审。
3. 实施严格的产品控制。
4. 采用现代程序设计技术。
5. 结果应能清楚地审查。
6. 开发小组的成员应该少而精。
7. 不断改进软件工程实践的必要性。

4.3 软件开发方法概述 [p.78–79]

软件开发方法就是使用定义好的技术（集）及表示符号来组织软件生产过程的方法 [p.78]。包括：

- 开发步骤（包括每步相应的技术和符号）
- 软件文档格式
- 开发方案评价标准

Niklaus Wirth 提出的经典公式 [p.79]：**程序 = 数据结构 + 算法**。

基于这个公式，不同的软件开发方法从不同角度看待程序：

- **结构化方法（Structured Method）**：从功能（算法）角度看程序，以功能为中心。
- **面向对象方法（Object-Oriented Method）**：从数据结构（数据）角度看程序，以数据为中心。
- **面向方面方法（Aspect-Oriented Method）**：解决结构化方法或面向对象方法的横切关注点问题。
- **形式化方法（Formal Method）**：用数学的方法分析并构建程序。

4.4 结构化方法（Structured Method） [p.81–85]

4.4.1 核心思想 [p.81]

结构化方法认为系统是由一些功能的相互联系、相互作用而形成 [p.81]。基本特征包括：

- 自顶向下（Top-Down）、逐步求精（Stepwise Refinement）
- 单入口和单出口（Single Entry, Single Exit）
- 利用抽象（Abstraction）和功能分解（Functional Decomposition）来处理软件复杂性问题

发展顺序：结构化程序设计（Structured Programming）→ 结构化设计（Structured Design）→ 结构化分析（Structured Analysis） [p.81]。

代表性方法：面向数据流图的方法、IDEF0 方法、Jackson 方法、LCP 方法等 [p.81]。

优点 [p.81]：方法简单实用、技术成熟、应用广泛。

不足 [p.81]：

- 不适应规模大的项目及特别复杂的项目。
- 难于解决软件重用（复用）问题。
- 难于适应需求变化的问题。
- 软件维护依然比较复杂。

4.4.2 结构化方法的起源：Dijkstra 的贡献 [p.82]

1968 年 Dijkstra 给“ACM 通讯”写了一篇短文 *Go To Statement Considered Harmful*, 提出 Go To 语句应从一切高级语言中去掉, 只用三种基本控制结构就可以写各种程序 [p.82]。

Dijkstra 被西方学术界称为“结构程序设计之父”, 其主要贡献 [p.82]:

- 提出“goto 有害论”。
- 提出信号量 (Semaphore) 和 PV 原语。
- 解决了“哲学家聚餐”问题。
- Dijkstra 最短路径算法和银行家算法 (Banker's Algorithm) 的创造者。
- 第一个 Algol 60 编译器的设计者和实现者。
- THE 操作系统的设计者和开发者。

4.4.3 结构化方法发展史 [p.83]

重要里程碑 [p.83]:

- 1971 年: Niklaus Wirth 提出逐步设计 (Stepwise Design)。
- 1972 年: Nassi-Shneiderman 图 (N-S 图) 的出现。
- 1974 年: Warnier/Orr 图、HIPO (层次化输入-处理-输出图)。
- 1975 年: Jackson 结构化程序设计、结构化设计 (Constantine, Yourdon, Stevens)。
- 1978 年: 结构化分析 (DeMarco, Yourdon)。
- 1979 年: Tom DeMarco 的 *Structured Analysis and System Specification*。
- 1983 年: 英国商务办公室提出 SSADM (结构化系统分析及设计方法)。
- 1985 年: IDEF0 (以 SADT 为基础)。
- 1988 年: Hatley-Pirbhai 模型 (实时系统规格)。
- 1990 年代: 信息工程 (Information Engineering, Clive Finkelstein / James Martin)。

4.4.4 结构定理 (Böhm-Jacopini 理论) [p.84]

1966 年, Corrado Böhm 和 Guiseppe Jacopini 证明了**结构定理**: 任何逻辑问题均可仅用顺序 (Sequence)、选择 (Selection) 和循环 (Iteration) 三种结构来解决 [p.84]。

值得注意的是, 1965 年, 中国唐稚松院士在《数学学报》发表了一篇关于计算机指令系统性质的论文, 提出转移指令可用循环替代, 比 Böhm-Jacopini 定理早一年 [p.84]。

20 世纪 80 年代采用结构程序设计方法的两个著名项目 [p.84]:

- 纽约时报信息库管理系统: 含 8.3 万行源代码, 只花了 1 年。
- 美国宇航局空间实验室操作的模拟系统: 含 40 万行源代码, 只用两年时间完成。

4.5 面向对象方法 (Object-Oriented Method) [p.86–90]

4.5.1 核心思想 [p.86]

面向对象方法认为系统是由一些对象的相互联系、相互作用而形成 [p.86]。其核心是尽可能模拟人类习惯的思维方式，使开发软件的方法与过程尽可能接近人类认识世界的方法与过程。

四大核心概念：对象 (Object) + 类 (Class) + 继承 (Inheritance) + 消息通信 (Message Communication)。

发展顺序：面向对象程序设计 (OOP) → 面向对象设计 (OOD) → 面向对象分析 (OOA) [p.86]。

代表性方法：Coad/Yourdon 方法、Booch 方法、OMT 方法、OOSE 方法、IDEF4 方法、CRC 方法等 [p.86]。

优点 [p.86]：

- 对象与功能相比，对象更易理解。
- 由对象建立的系统往往更稳定。
- 面向对象方法易学习。
- 面向对象方法易处理需求模糊或变化的情况。
- 由对象建立的系统便于重用 (Reuse)。
- 类的封装使系统易于修改维护。

4.5.2 面向对象方法的不足 [p.87]

Erlang 语言发明者 Joe Armstrong 的观点 “Why OO Sucks” [p.87]：

- 类作为复用单元，有时显得太小。
- 继承会增加类间的耦合性 (Coupling)。
- 面向对象方法理论没有结构化方法成熟。
- 面向对象方法比结构化方法复杂。

4.5.3 面向对象方法的起源 [p.88]

- **60 年代中后期**：Simula 语言在 ALGOL60 基础上研制开发，提出了对象的概念，并使用了类，也支持类继承。Simula67 作者 Ole-Johan Dahl 和 Kristen Nygaard 于 2001 年获图灵奖。
- **70 年代**：Smalltalk 语言诞生，取 Simula 的类为核心概念，在系统设计中强调对象概念的统一，引入对象、对象类、方法、实例等概念和术语，采用动态联编和单继承机制。Smalltalk 作者 Alan Kay 于 2003 年获图灵奖。

重要评价：面向对象源出于 Simula67，真正的 OOP 由 Smalltalk 奠基。Smalltalk 现在被认为是最纯的面向对象编程语言 (OOPL) [p.88]。

4.5.4 问题空间与软件空间一致 [p.89]

面向对象方法强调运用人类日常的逻辑思维方法与原则（如抽象、分类、继承、聚合、封装、关联等）。这使得软件开发能更有效地分析和解决问题 [p.89]。

关键优势：无论系统的构成成分还是它们之间的关系体现的系统结构，都可直接映射问题域。问题域的成分相对稳定，这样的映射一方面可以使系统的软件结构与所要解决的问题结构保持一致，另一方面又可使系统的软件结构相对稳定 [p.89]。

4.6 面向方面方法（Aspect-Oriented Method） [p.91–96]

4.6.1 横切关注点（Crosscutting Concerns） [p.91–92]

横切关注点是散布在功能模块或对象类中的横切行为 [p.91]：

- 结构化方法以功能为中心，当数据发生变化时需修改许多功能模块。
- 面向对象方法以对象为中心（核心是数据），当需要改变某一功能时，可能需要修改许多对象。
- 面向方面方法利用关注点分离（**Separation of Concerns**）来处理软件复杂性——每个成分（类、方法、过程等）做且只做一件事（一个关注点） [p.92]。

关注点（Concern）：可以定义为一个或一组项目信息持有者感兴趣的或对他们意义重大的事情。例如，在不同的系统功能中记录操作日志。

方面（Aspect）：是指涉及到若干功能模块或对象类的系统功能（即横切关注点） [p.92]。

4.6.2 面向方面程序设计的一般模型 [p.93]

核心要素：

- **切入点（Pointcut）**：连接点（Join Point）的集合。连接点是系统执行过程中明确定义的一个事件，是横切行为的插入点。
- **类间声明和通知（Advice）**：表示横切行为。类间声明用于在已有类型中添加新的方法、属性、接口等，而通知表明在切入点处增加的行为。

4.6.3 关注点的五类划分（Ian Sommerville） [p.94]

1. **功能性关注点（Functional Concerns）**：与特殊功能相关的关注点。系统的核心关注点是系统主要目标的功能性关注点，例如火车控制系统中的火车制动。
2. **服务质量关注点（Quality of Service Concerns）**：与非功能性行为相关的关注点，包括性能（Performance）、可靠性（Reliability）和可用性（Availability）等。
3. **政策关注点（Policy Concerns）**：与管理系统使用的总政策相关的关注点，包括信息安全关注点、业务规则相关的关注点等。
4. **系统关注点（System Concerns）**：与系统总体属性相关的关注点，例如系统的可维护性和可配置能力。

5. **机构关注点 (Organizational Concerns)**: 与机构目标和优先级相关的关注点, 例如对现存软件资产的利用等。

4.6.4 面向方面方法概述 [p.95]

面向方面方法认为, 一个系统是由核心关注点构成的核心系统和若干横切关注点构成的若干方面扩展所构成 [p.95]。代表性方法: Jacobson 提出的 AOSD 方法、Clark 提出的 Theme 方法等。

4.7 形式化方法 (Formal Method) [p.97–100]

4.7.1 核心思想 [p.97]

形式化方法认为系统可以通过严格的、规范化的数学理论经分析、推理和转换得到 [p.97]。

代表性方法: VDM (Vienna Development Method, 维也纳开发方法)、RAISE 方法、Event-B 方法 [p.97]。

优点 [p.97]:

- 形式模型完整、一致和无二义性。
- 支持形式推理, 便于软件验证。
- 便于软件自动生成。

不足 [p.97]:

- 一般人不易接受, 需要培训, 增加成本。
- 灵活性差。
- 难以与工程化的软件开发过程平滑地结合。
- 支持工具少。

4.7.2 发展历史 [p.98–99]

最早可追溯到 20 世纪 50 年代的编译技术研究 (BNF 自动生成程序分析工具) [p.98]。20 世纪 60 年代后期, 出现了一系列形式化方法: Petri 网方法、VDM/VDM++、CSP/CCS、Z/B/Event-B 等 [p.98]。

关键人物 [p.98]: Petri (Petri 网)、Hoare (CSP, Communicating Sequential Processes)、Milner (CCS, Calculus of Communicating Systems)、Abrial (B 方法/Event-B)。

形式化方法的两大研究内容 [p.99]:

1. **形式规约 (Formal Specification)**: 对程序“做什么”的精确数学描述 (描述做什么, 不描述如何做)。
2. **形式验证 (Formal Verification)**: 正确性证明 (Correctness Proof)、模型检测 (Model Checking)、模拟 (Simulation)。

4.8 多方法集成：SOFL 方法 [p.101–103]

利用各种方法的长处，实现优势互补 [p.101]。S. Liu（刘少英，日本法政大学）等人提出了 SOFL（Structured Object-based Formal Language）语言及其开发方法学，集成了结构化方法、面向对象方法和形式化方法 [p.101]。

SOFL 方法的特点 [p.101]：

- 用结构化方法进行需求分析和规格说明。
- 用面向对象方法进行设计和实现。
- 在结构化和面向对象的开发过程中，形式化方法用于提供高质量的规格说明和各级系统的验证。

后续发展 [p.103]：Agile-SOFL 开发过程，将敏捷（Agile）思想与形式化工程方法相结合，提出了 Agile Formal Engineering Method for Software Productivity and Reliability。

5 软件工具与集成开发环境 (Software Tools and Integrated Development Environments)

5.1 软件工具 [p.108–112]

对一个待开发的系统，先考虑采用何种方法（看待系统的立场、观点等），然后再考虑采用何种工具（提高开发质量和效率）[p.108]。

软件工具 (Software Tool) 是指为支持计算机软件的开发、维护及有关工作而研制的程序系统。目的是降低软件开发和维护的成本，提高软件产品的生产效率和质量 [p.108]。

软件工具的分类 [p.108–111]：

1. **软件开发工具** [p.109]：需求分析工具、软件设计工具、软件编码工具、文档写作工具、软件测试工具。
2. **软件维护工具** [p.110]：版本控制工具、文档分析工具、逆向工程工具、再工程工具。
3. **项目管理工具** [p.111]：项目管理工具、配置管理工具、软件评价工具、开发信息库工具。

5.1.1 软件工具的局限性 [p.112]

- 一般情况一种软件工具只支持一种活动（软件开发和维护过程中进行的活动较多）。
- 工具界面不统一，工具内部无联系，工具切换由人工操作。
- 对大型软件的开发和维护的支持能力受限。

5.2 集成型软件开发环境 [p.114–116]

集成型软件开发环境 (Integrated Software Development Environment) 由软件工具集和环境集成机制构成 [p.114]：

- **软件工具集**：用以支持软件开发的相关过程、活动和任务（支持某种开发方法）。
- **环境集成机制**：为工具集成和软件开发、维护和管理提供统一的支持。

5.2.1 集成化项目支持环境 (IPSE) [p.115]

IPSE (Integrated Project Support Environment) 分层结构 [p.115]：

- **核心层**：宿主机硬件和操作系统（如 UNIX）。
- **基本层**：各种语言工具（编译程序、Shell 语言、装入程序等）、环境数据库或文件、测试分析工具、配置管理工具、用户界面等。
- **应用层**：各种方法开发工具（支持软件工程各种方法学），如快速原型开发工具、需求分析工具等。

5.2.2 ECMA 软件开发环境参考模型 [p.116]

ECMA 参考模型定义了软件开发环境的功能组件 [p.116]:

- 数据存储服务 (Data Repository Services)
- 数据集成服务 (Data Integration Services)
- 任务管理服务 (Task Management Services)
- 消息服务 (Message Services)
- 用户界面服务 (User Interface Services)
- 工具槽 (Tool Slots)

5.3 AI 辅助软件工具 [p.118–127]

5.3.1 大模型如何使能软件工程 [p.118]

2024 年度中国计算机大会探讨了大模型如何使能软件工程 [p.118]。

5.3.2 生成式 AI 核心技术 [p.119]

三大核心模型架构 [p.119]:

- **生成对抗网络 (GAN, Generative Adversarial Network)**: 由生成器 (Generator) 和判别器 (Discriminator) 组成, 两者相互对抗训练。
- **Transformer 模型**: 基于注意力机制 (Attention Mechanism), 是现代大语言模型的基础架构。
- **扩散模型 (Diffusion Model)**: 包括正向加噪 (Forward Diffusion) 和逆向除噪 (Reverse Denoising) 两个过程。

5.3.3 AI 编程工具 [p.121]

当前主流 AI 编程助手 [p.121]:

- 豆包 Marscode: <https://www.marscode.cn/workbench>
- 通义灵码: <https://lingma.aliyun.com>
- DeepSeek: <https://chat.deepseek.com/>
- 腾讯云 Copilot: <https://copilot.tencent.com/>

5.3.4 百度千帆 API 调用示例 [p.123–125]

获取 API Key 步骤 [p.123]:

1. 进入百度千帆 ModelBuilder 平台，注册并登录。
2. 在百度智能云管理中心页面创建 API Key。
3. `pip install openai` 安装 OpenAI SDK。

支持的模型包括 DeepSeek-V3、DeepSeek-R1 等 [p.123]。

5.3.5 AI 辅助编码环境搭建 [p.126]

VSCode + Cline + DeepSeek 组合搭建 AI 辅助编码环境 [p.126]。

5.4 Flask Web 框架实战 [p.128–193]

5.4.1 Flask 框架特点 [p.128]

Flask 是一个用 Python 编写的轻量级 Web 框架 [p.128]:

- Micro-framework（小型框架）。
- Extensible framework（可扩展框架）。
- MVC 架构（Model-View-Controller）。
- 两大核心依赖：Werkzeug（路由等）和 Jinja2（模板等），均由 Poccoo 开发。

5.4.2 环境搭建 [p.130]

```
# 安装virtualenv
pip install virtualenv
# 创建虚拟环境
mkdir 项目目录
cd 项目目录
virtualenv venv
# 激活环境
venv/bin/activate          # Linux
venv/scripts/common/activate # Windows
# 安装Flask
pip install Flask
```

5.4.3 第一个 Flask 应用 [p.131]

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World'

if __name__ == '__main__':
    app.run()
```

要点解析：Flask 类实例化创建应用；`@app.route('/')` 装饰器告诉应用哪个 URL 应该调用关联的函数（视图函数）；`app.run()` 在本地开发服务器上运行应用 [p.131]。

5.4.4 项目目录结构 [p.133]

```
项目/
|-- app/                    # 程序包
|   |-- templates/         # Jinja2模板
|   |   |-- common/       # 通用模板
|   |   |-- errors/       # 错误页面
|   |   |-- user/         # 用户模板
|   |   |-- email/        # 邮件发送
|   |-- static/            # 静态文件
|   |   |-- js/           # JS脚本
|   |   |-- css/          # 样式表
|   |   |-- image/        # 图片
|   |   |-- favicon.ico   # 网站图标
|   |-- main/              # 主程序包
|       |-- __init__.py
|       |-- errors.py
|       |-- forms/         # 表单处理
|       |-- views/         # 视图处理
|       |-- models/        # 数据库模型
|-- migrations/            # 数据迁移文件夹
|-- tests/                 # 单元测试
|-- venv/                  # 虚拟环境
|-- requirements.txt       # 依赖包列表
|-- config.py              # 全局配置
|-- manage.py              # 启动程序
```

5.4.5 配置管理 [p.134]

Flask 支持多环境配置 [p.134]:

- DevelopmentConfig (开发环境): DEBUG = True, SQLite 数据库。
- TestingConfig (测试环境): TESTING = True。
- ProductionConfig (生产环境): 使用环境变量配置数据库。

5.4.6 依赖管理 [p.135]

生成依赖文件

```
(venv) $ pip freeze > requirements.txt
```

从依赖文件安装

```
(venv) $ pip install -r requirements.txt
```

5.4.7 路由机制 [p.138–139]

Flask 类中支持路由功能的数据结构 [p.138]:

- url_map (Map 类型): 建立 URL 到 endpoint 的映射, 实现动态路由功能。
- view_functions (字典): 建立 endpoint 和视图函数之间的映射关系。
- Map 和 Rule 是 Werkzeug 中实现的映射类和路由类。

5.4.8 动态路由 [p.140]

支持参数类型: string、int、float。如果没有指定参数类型, 默认为 string [p.140]:

```
@app.route('/<int:id>/')
def h(id):
    id = id ** 8
    return str(id)
```

5.4.9 HTTP 请求方法 [p.141]

- GET: 向特定资源发出请求。
- POST: 向指定资源提交数据进行处理 (提交表单、上传文件)。
- PUT: 向指定资源位置上传最新内容。
- DELETE: 请求服务器删除指定资源。
- TRACE: 回显服务器收到的请求 (测试或诊断)。

5.4.10 文件上传 [p.142–143]

使用 Flask 处理文件上传的核心步骤:使用 `request.files` 获取文件,使用 `secure_filename` 处理文件名安全,使用 HTML 表单的 `enctype='multipart/form-data'` 和 POST 方法 [p.142]。

5.4.11 模板 (Templates) 与 Jinja2 [p.145–154]

模板是包含响应文本的文件,其中包含占位变量用于动态部分。将变量替换为实际值并返回最终响应字符串的过程称为渲染 (Rendering) [p.146]。

Jinja2 模板引擎特性:

- 变量: `{{ variable }}` 结构引用变量 [p.148]。
- 过滤器 (Filters): 通过管道符 `|` 附加在变量名后修改变量,如 `{{ name|upper }}` [p.149]。
- 条件语句: `{% if ... %} ... {% else %} ... {% endif %}` [p.151]。
- 循环语句: `{% for ... in ... %} ... {% endfor %}`, 支持 loop 变量 (`index`、`first`、`last`、`length` 等) [p.152]。
- 宏 (Macros): 类似 Python 函数,可存储在独立文件中通过 `{% import ... %}` 导入 [p.153]。
- 模板继承: 使用 `{% extends "base.html" %}` 和 `{% block ... %}` 实现 [p.154]。
- 模板包含: 使用 `{% include ... %}` [p.154]。

5.4.12 Bootstrap 集成 [p.155–158]

Bootstrap 是 Twitter 开发的开源前端框架,提供整洁且具有吸引力的网页 UI 组件 [p.155]。
安装与使用:

```
pip install flask-bootstrap
from flask_bootstrap import Bootstrap
bootstrap = Bootstrap(app)
```

使用模板继承 Bootstrap 基础模板 [p.157]:

```
{% extends "bootstrap/base.html" %}
{% block title %}Flasky{% endblock %}
{% block content %}
<div class="container">
  <div class="page-header">
    <h1>Hello, {{ username }}!</h1>
  </div>
</div>
{% endblock %}
```

5.4.13 Favicon 设置 [p.159]

Favicon 是被浏览器用于标识书签网站的小图标，显示大小为 16×16 像素，文件扩展名为“.ico” [p.159]。

5.4.14 Web 表单 (Flask-WTF) [p.161–167]

表单通过 POST 请求将数据由 Web 浏览器提交到服务器，服务器端通过 `request.form` 访问表单数据 [p.161]。

配置 [p.162]：需要在 `config.py` 中设置 `CSRF_ENABLED = True` 和 `SECRET_KEY`。

表单定义 [p.163]：

```
from flask_wtf import FlaskForm as Form
from wtforms import StringField, PasswordField
from wtforms.validators import DataRequired

class LoginForm(Form):
    accountNumber = StringField('accountNumber',
                                validators=[DataRequired('accountNumber is null')])
    password = PasswordField('password',
                              validators=[DataRequired('password is null')])
```

5.4.15 数据库操作：Flask-SQLAlchemy [p.168–192]

Flask-SQLAlchemy 是 Flask 的扩展，是对 SQLAlchemy 的进一步封装 [p.168]。

SQLAlchemy 是一个基于 Python 实现的 ORM (Object-Relational Mapping) 框架，将类和对象转换成 SQL。其优势包括 [p.168]：

- 大大缩短程序员的编码时间。
- 良好的数据库操作接口，代码简洁易读、风格统一。
- 动态的数据表映射，减少代码修改。
- 减少程序员对数据库的学习成本。
- 可以方便地引入数据缓存等附加功能。

数据库配置 [p.169–170]

```
app.config['SQLALCHEMY_DATABASE_URI'] =
    'mysql+pymysql://root:root@localhost:3306/student'
```

重要配置项 [p.170]：

- `SQLALCHEMY_COMMIT_ON_TEARDOWN`：设为 `True` 时，每次请求结束后自动提交数据库变动。
- `SQLALCHEMY_TRACK_MODIFICATIONS`：设为 `False` 时，减少内存使用。

MySQL 数据库操作 [p.171–174] 基本操作 [p.171–174]:

- 进入 MySQL: `mysql -h < 主机地址 > -u < 用户名 > -p < 密码 >`
- 创建用户: 使用 `INSERT`、`GRANT` 或 `CREATE USER` 语句。
- 创建数据库: `create database < 数据库名 >;`
- 创建表: `CREATE TABLE students (...)`
- 显示表结构: `desc < 表名 >;`

ORM 模型定义 [p.175–176]

```
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy(app)

class Role(db.Model):
    __tablename__ = 'roles'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(64), unique=True)
    users = db.relationship('User', backref='role')

class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True, index=True)
    email = db.Column(db.String(128), unique=False, index=True)
    role_id = db.Column(db.Integer, db.ForeignKey('roles.id'))
```

数据库查询 [p.177–178] 常用查询操作 [p.177]:

- 全查询: `data = User.query.all()`
- 过滤查询: `User.query.filter(User.username=='wangli', User.age==15)`
- 单条件: `User.query.filter_by(age=18)`
- 模糊查询: `User.query.filter(User.username.like('%wang%'))`
- 逻辑查询 [p.178]: 使用 `and_`, `or_`, `not_`。

MTV 架构 [p.180] 示例应用采用 MTV (Model-Template-View) 架构 [p.180]:

浏览器 → URL 调度 → 视图 (View) → 模型 (SQLAlchemy ORM) → 数据库;
视图 → 模板 (Template) → 浏览器。

完整应用示例 [p.181–193] 项目目录: StudentMicroblog, 应用目录: studentapp [p.181]。核心文件包括 [p.181–191]:

- config.py: 配置文件 (数据库连接、密钥等)。
- model.py: ORM 模型定义 (User, Post)。
- mystudentapp.py: 应用初始化。
- studentForm.py: 表单定义 (LoginForm)。
- view.py: 视图函数 (路由处理、业务逻辑)。
- server.py: 启动程序。
- templates/: 模板文件 (base.html, home.html, login.html)。

Flask Shell 操作 [p.184–185] 可以通过 Flask Shell 交互操作 ORM [p.184–185]:

```
$ set FLASK_ENV=development
$ flask --app server.py shell
>>> from mystudentapp import db
>>> from model import User, Post
>>> db.create_all()
>>> us1 = User(nickname='liming', pwd='123456')
>>> db.session.add(us1)
>>> db.session.commit()
```

6 基于 Trae 的软件开发 (Trae-based Software Development)

6.1 Trae 工具简介 [p.194–199]

Trae (The Real AI Engineer) 是字节跳动于 2025 年 1 月 19 日发布的一款面向专业开发者提供服务的全新 AI Coding 产品 [p.197]。2025 年 11 月 12 日, Trae 宣布推出 SOLO 正式版; Skills 功能于 2026 年 1 月 15 日正式上线 [p.197]。

官方网站: <https://www.trae.cn/>。

Trae 的核心能力 [p.197]:

- **需求理解:** 支持自然语言输入, 能解析模糊、复杂甚至矛盾的需求。
- **工具调度:** 自动调用代码生成器、调试器、测试框架、Docker、Git 等工具。
- **自主执行:** 在 SOLO 模式下, 可独立完成端到端开发任务。

6.1.1 Trae 的安装与基本使用 [p.198–199]

下载网址: <https://www.trae.cn/>。注册后即可免费使用所有功能 [p.198]。

基本使用流程 [p.199]:

1. 创建项目文件夹。
2. 进入 Trae, 打开该文件夹。
3. 写 Prompt (提示词)。例如: “你是一个 C++ 程序员, 采用 C++20 完成一个整型数组元素的快速排序, 要求数组大小不超过 20。”
4. Trae 思考并生成代码。
5. Trae 进行项目测试。
6. 改进代码。例如: 提示 “提高模块 quicksort.cpp 的可读性”。

6.2 Trae 智能体简介 [p.200–205]

Trae 内置了四个智能体 (Agent) [p.201]:

1. **Chat 智能体** [p.202]: 可以检查代码库或编写代码, 但不能生成代码文件。其它与普通的 AI 聊天窗口一样。
2. **Builder 智能体** [p.203]: 可以对文件进行编辑、调用命令行终端、预览和联网搜索。
3. **Builder with MCP 智能体** [p.204]: 在 Builder 基础上增加了 MCP (Model Context Protocol) 调用功能。MCP 是由 Anthropic 在 2024 年推出的开源协议, 旨在实现大语言模型与外部数据源和工具的集成, 通过相同的协议同时处理本地资源 (数据库、文件、服务等) 和远程资源 (GitHub 等 API)。

4. **SOLO Coder 智能体** [p.205]: 面向复杂项目开发的智能体, 助力高效完成从需求迭代到架构重构的全流程开发工作。SOLO 模式提供一系列工具 (编辑器、文档、浏览器等), 可添加自定义模型。

6.3 Skill 创建与使用 [p.206–213]

6.3.1 Skill 简介 [p.207]

技能 (Skill) 可以被视为提供给智能体的一套 “专业能力说明书” (类似用户手册或操作指南 SOP)。在执行任务时, 智能体可以按需加载相应的技能, 从而增强其对任务的理解与执行能力 [p.207]。

关键公式: 智能体 = 大模型 (大脑) + 工具 (手脚) + Skills (灵魂) [p.207]。

Skill 的核心特点 [p.207]:

- 仅支持在 SOLO 模式中使用。
- 仅在当前项目内生效。
- 以结构化方式描述完成某一类任务所需的信息。
- 动态按需加载。

6.3.2 Skill 使用场景 [p.208]

- **高频重复操作**: 同一个指令已手动输入过很多次。
- **要求输出一致**: 需要跨越多次对话保持相同的输出风格、格式或标准。
- **固定的工作流程**: 有一套明确的多步骤工作流程需要严格执行。
- **沉淀专业知识**: 把团队的最佳实践 (代码规范、品牌指南、测试流程、数据分析方法等) 固化下来, 让 AI 也能掌握。

6.3.3 Skill 创建示例 [p.209–210]

在 SOLO 模式下, 用提示词创建 Skill [p.209]:

你是 C++ 编程专家, 请编写一个 skill, 检查 C++ 程序是否满足 Google 关于 C++ 代码规范的要求。

Trae 会自动在 `.trae/skills/cpp-google-style-checker/` 目录中创建 SKILL.md 文件, 并提示用法 [p.209]:

- 安装相关依赖: `pip install cpplint 和 clang-format`。
- 检查文件: `cpplint path/to/file.cpp`。
- 自动格式化: `clang-format -i -style=Google path/to/file.cpp`。

6.3.4 Skill 使用 [p.211–212]

将 Skill 名称改为“CPP 风格检查大师”，然后输入提示词 [p.211]：

使用“CPP 风格检查大师”对 quicksort.cpp 进行检查。

Trae 会依据 SKILL.md 的描述安装 CPPLint，用户只需确认即可。随后 Trae 会对程序进行检查，指出问题，并对程序进行修改 [p.212]。

6.3.5 Skill 文件结构 [p.213]

Skill 本质上是一个包含了 SKILL.md 文件的文件夹 [p.213]：

```
my-skill/
|-- SKILL.md           # 核心提示模板
|-- scripts/          # 可执行Python/Bash脚本
|   |-- validator.py
|   |-- processor.sh
|-- references/        # 参考文档
|   |-- terminology.md
|   |-- examples.md
|-- assets/            # 模板和静态资源
|   |-- template.docx
|   |-- config.json
```

6.4 Rule 创建与使用 [p.214–224]

6.4.1 Rule 简介 [p.215]

Rules 是给 Trae AI 功能生成结果添加规则和限制。Rule 的优先级遵循：用户输入 > 自定义 Agent Prompt > user_rules.md > project_rules.md [p.215]。

Rule 的主要功能 [p.215]：

- 约束代码风格（如强制用驼峰命名、要求方法写注释等）。
- 限定技术选型（如指定优先使用某技术/框架/库，禁止使用某组件/框架/库）。
- 提前指定配置参数（如提前设置连接数据库方式、账号密码等）。

Rule 的三大应用场景 [p.215]：

1. **提升效率**：将个人经验和项目要求转化为可复用的规则，一次配置可长期生效。
2. **统一标准**：将团队规范、项目标准结构化为规则，保持风格、结构和质量上一致。
3. **保障质量**：让 AI 明确项目的核心约束（架构设计、命名规范、代码风格等），避免常见错误。

6.4.2 个人 Rule [p.216–218]

个人规则是基于个人使用习惯和需求为 AI 定制的规则，在所有项目中生效，且不会被 git 追踪 [p.216]。存放路径：C:\Users\rsb\.trae-cn\user_rules。

个人规则示例 [p.218]：

- 语言与沟通：保持对话语言为中文，代码注释使用中文，技术术语保持英文。
- 代码执行规则：不主动运行代码，修改前先查看文件，确认命令的正确性和安全性。
- 代码风格：遵循项目现有风格，优先使用已有工具库和组件，保持代码简洁，复杂逻辑添加注释。

6.4.3 项目 Rule [p.219–221]

项目规则是应用在单个项目中且会被 git 追踪的规则，规则文件位于项目的 .trae/rules 目录 [p.219]。内容包括 [p.219]：

- 代码风格：缩进、命名规范等。
- 语言与框架：使用的编程语言或框架。
- API 限定：明确禁止或限制使用的 API。
- 技术选型：限定技术栈。

6.4.4 Rule 最佳实践 [p.222–224]

1. 控制单条规则的内容粒度，避免在一段规则中包含过多信息，使其保持清晰、聚焦、易于理解 [p.222]。
2. 各条规则之间不得彼此冲突或相互覆盖 [p.222]。
3. 在指定文件路径时，使用相对于项目根目录的相对路径，以确保 AI 能准确定位文件 [p.222]。
4. 引用规则时，优先选择与当前对话或任务强相关的规则 [p.222]。
5. 新建或修改规则后，建议开启全新的对话再使用，以避免历史上下文与新规则产生冲突 [p.222]。
6. 若项目中已有大量不符合规范的代码，模型可能会沿用现有代码风格而非遵循新规则 [p.222]。

通用编码与代码可读性实践 [p.224]：

- 避免不必要的对象复制或克隆；避免多层嵌套，提前返回；使用适当的并发控制机制。
- 注释应该解释“为什么”而不是“做什么”；使用有意义的、描述性的名称；遵循项目或语言的命名规范；相关代码放在一起；函数只做一件事。

7 附录：核心概念速查表

术语（中文）	术语（English）及简要说明
系统	System: 由要素组成的具有特定功能、结构和环境的整体
系统工程	Systems Engineering: 合理开发、管理、运行和改进复杂系统的理论和方法
软件	Software: 以程序、数据和文档形式出现的计算机逻辑部分
软件危机	Software Crisis: 软件质量差、成本超支、进度拖延等问题
软件工程	Software Engineering: 用工程化方法开发高质量软件的学科
可信性	Dependability: 系统提供预期服务水平的能力
可靠性	Reliability: 给定条件下持续提供正确服务的概率度量 $R(t)$
可用性	Availability: t 时刻系统能正确运行的概率,稳态可用度 $A = MTTF/(MTTF + MTTR)$
防危性	Safety: 单位时间内失效损失期望值小于规定阈值
完整性	Integrity: 不存在未经授权修改或操控
安全性	Security: 机密性 + 完整性 + 可用性（CIA 三要素）
可维护性	Maintainability: 失效后可修复的概率 $M(t) = 1 - e^{-\mu t}$
MTTF	Mean Time To Failure: 平均失效等待时间
MTTR	Mean Time To Repair: 平均失效修复时间
拜占庭故障	Byzantine Fault: 行为完全无法预测的故障
CIA 三要素	Confidentiality, Integrity, Availability
结构化方法	Structured Method: 以功能为中心的软件开发方法
面向对象方法	Object-Oriented Method: 以数据为中心，基于对象 + 类 + 继承 + 消息通信
面向方面方法	Aspect-Oriented Method: 利用关注点分离处理横切行为
形式化方法	Formal Method: 基于严格数学理论的软件开发方法
横切关注点	Crosscutting Concerns: 散布在多个模块中的横切行为
ORM	Object-Relational Mapping: 对象关系映射
MTV 架构	Model-Template-View: Flask 的 Web 应用架构模式
MCP	Model Context Protocol: 大模型与外部工具的标准化连接协议
Trae	The Real AI Engineer: 字节跳动的 AI Coding 产品
SQL O-Code	面向复杂项目开发 的 Trae 智能体